

Prueba técnica Erco

Explicación de las decisiones de diseño tomadas y posibles mejoras futuras

Arquitectura general

El proyecto carga las mediciones de energía en PostgreSQL y expone el cálculo de la factura mediante una API. Todo corre con Docker Compose en tres servicios:

Servicio	Puerto	Función
PostgreSQL 17	5432	Base de datos `erco` con las tablas y vistas
Airflow 3	8080	Carga diaria de los archivos fuente (CSV) a la base
API FastAPI	8000	Cálculo de la factura y estadísticas

El flujo es: archivos CSV, Airflow, PostgreSQL y, al final, la API, que lee la base y responde en JSON.

Base de datos

Usuarios con permisos separados, para mantener la seguridad

Se crearon dos usuarios en PostgreSQL, cada uno con solo los permisos que necesita:

- `erco_escritura`: crea las tablas y carga los datos. Lo usa Airflow.
- `erco_lectura`: solo puede consultar las vistas; no puede leer las tablas directamente, modificar datos ni crear objetos. Lo usa la API.

Así, si un componente falla o se ve comprometido, el daño queda limitado a lo que puede hacer su usuario. Por ejemplo, la API no puede modificar ni borrar datos aunque tenga un error.

Modelo de tablas

Se implementaron las seis tablas del diagrama: `services`, `records`, `consumption`, `injection`, `tariffs` y `xm_data_hourly_per_agent`.

- Cada tabla tiene una llave primaria numérica autoincremental, y su llave natural (por ejemplo `id_service` o `id_record`) es un índice único. Esto impide registros duplicados y es la base de la carga sin duplicados.
- Las llaves foráneas garantizan la integridad: cada medición (`records`) pertenece a un servicio existente, y cada consumo o inyección pertenece a una medición existente.
- Restricciones `CHECK`: el nivel de tensión debe estar entre 1 y 4, y en las tarifas de nivel 2 y 3 el cdi debe ser nulo, porque el documento indica que en esos niveles no aplica.
- Índices pensados para las consultas de la API, como `(id_service, record_timestamp)` para buscar las mediciones de un servicio en un mes.
- Los valores de energía y tarifas usan el tipo `numeric`, que es exacto, en lugar de `float`.

Columnas de control, para controlar los registros en caso de falla

Cada tabla tiene columnas de control:

- `created_at` y `created_by`: cuándo y quién creó el registro.
- `updated_at` y `updated_by`: cuándo y quién lo modificó por última vez.
- `source_file`: de qué archivo se cargó.

Si una carga falla o trae datos incorrectos, estas columnas permiten identificar qué registros se crearon o modificaron, en qué momento y desde qué archivo, para revisarlos o corregirlos. Las fechas se guardan en hora de Colombia.

Vistas

Cada tabla tiene una vista (`vw_services`, `vw_records`, etc.) que muestra solo las columnas de negocio, sin las de control. El usuario de lectura únicamente tiene acceso a estas vistas.

Archivos SQL

El esquema está en un único script SQL plano, `erco.sql`, que se ejecuta con un solo comando y crea todo en orden:

1. Los usuarios `erco_escritura` y `erco_lectura`.
2. La base `erco`, con la zona horaria de Colombia, y sus permisos.
3. Las seis tablas, en el orden que exigen las llaves foráneas (services, records, consumption, injection, tariffs y xm_data_hourly_per_agent), cada una con sus índices, su vista y el permiso de lectura sobre la vista.

Las tablas se crean como el usuario `erco_escritura`, que queda como su dueño.

Carga de datos con Airflow, para mantener actualizadas las fuentes

La carga de los archivos fuente a la base se hace con Airflow: hay un DAG por tabla, programado todos los días a las 08:00, hora de Colombia. Cada DAG tiene tres tareas:

1. **Extraer:** lee el archivo CSV y verifica que tenga las columnas esperadas.
2. **Transformar:** convierte los tipos de datos y valida. Los errores graves detienen la carga: campos obligatorios vacíos, valores negativos, nivel de tensión inválido o un mismo servicio con dos mediciones en la misma hora. Los problemas corregibles solo generan una advertencia y se corrigen: registros duplicados (se conserva el último) o tarifas de nivel 2 y 3 que traen cdi.
3. **Cargar:** inserta los datos en la base.

La carga es idempotente: si se ejecuta de nuevo con los mismos datos no duplica registros, y solo actualiza los que cambiaron, dejando registro de cuándo y quién los actualizó. De esta forma la base se mantiene al día con las fuentes sin intervención manual, y cada ejecución queda registrada con sus logs.

API REST con FastAPI

Endpoints

Endpoint	Qué hace
`POST /calculate-invoice`	Calcula los cuatro conceptos de la factura (EA, EE1, EE2 y EC) de un servicio en un mes
`POST /calculate-ea`	Calcula solo la Energía Activa

Endpoint	Qué hace
`POST /calculate-ee1`	Calcula solo los Excedentes tipo 1
`POST /calculate-ee2`	Calcula solo los Excedentes tipo 2, con el detalle hora a hora
`POST /calculate-ec`	Calcula solo la Comercialización de Excedentes
`GET /client-statistics/{service_id}`	Total, promedio, máximo y mínimo horario de consumo e inyección de un servicio
`GET /system-load`	Carga del sistema por hora: el consumo de todos los servicios sumado en cada hora

Los endpoints de cálculo reciben el servicio, el año y el mes, por ejemplo `{"service_id": 2256, "year": 2023, "month": 9}`. La documentación se genera automáticamente y se puede probar en `/docs`.

Cálculo de los conceptos

Con C igual al consumo total del mes e I igual a la inyección total del mes:

Concepto	Cantidad (kWh)	Tarifa ($/kWh)
EA	C	CU
EC	I	C (componente de comercialización)
EE1	I si I es menor o igual que C; si no, C	CU negativo
EE2	0 si I es menor o igual que C; si no, I menos C	valor de XM de cada hora

EE2 se calcula hora a hora: se acumula la inyección en orden cronológico y, desde la hora en que supera el consumo total del mes, el exceso de cada hora se multiplica por el valor de `xm_data_hourly_per_agent` de esa hora. El cruce entre el servicio y su tarifa ignora el cdi en los niveles de tensión 2 y 3, como indica el documento.

Decisiones tomadas sobre el cálculo:

- La cantidad de EE2 es inyección menos consumo. El texto del documento dice "consumo menos inyección", pero eso da un valor negativo; el ejemplo del documento muestra 150 menos 100 igual a 50, y se siguió el ejemplo.
- La tarifa de EE2 se devuelve vacía, porque cambia hora a hora. El endpoint de EE2 entrega cada hora con su multiplicación, y la suma de esas horas es el monto.

- Los montos se calculan con el tipo `Decimal` de Python, que es exacto, para no perder centavos por redondeo.

Organización del código

El código está separado en capas, cada una con una sola responsabilidad:

- **Endpoints** (`main.py`): reciben la petición, validan los datos de entrada y convierten los errores en respuestas HTTP.
- **Facturación y estadísticas** (`facturacion.py`, `estadisticas.py`): obtienen los datos y aplican los cálculos.
- **Cálculos** (`calculos.py`): solo las fórmulas, sin acceso a la base de datos, lo que permite probarlas de forma aislada.
- **Acceso a datos** (`db.py`): todas las consultas SQL.

Manejo de errores y validaciones

Código	Cuándo
404	El servicio no existe, no tiene tarifa o no tiene mediciones en el mes pedido
422	Datos de entrada inválidos (por ejemplo, un mes fuera de 1 a 12) o falta el valor de XM de una hora con excedente
503	No hay conexión con la base de datos

Cada error responde con un mensaje que explica el motivo, por ejemplo "No existe el servicio 9999".

Optimización de consultas SQL

- Las sumas y agrupaciones se hacen en PostgreSQL, no en Python.
- Los filtros de fecha usan rangos (desde el primer día del mes hasta el primer día del mes siguiente), lo que permite usar el índice `(id_service, record_timestamp)`
- El detalle hora a hora solo se consulta cuando hay excedentes tipo 2.
- Cada petición usa una sola conexión para todas sus consultas.
- Las consultas usan parámetros, lo que evita la inyección SQL.

Posibles mejoras futuras

- Autenticación con token, para que solo las personas autorizadas puedan consultar las facturas de los clientes.

- Un pool de conexiones, para reutilizar las conexiones a la base en lugar de abrir una nueva en cada petición cuando aumente el número de consultas simultáneas.
- Filtros por periodo y paginación en `/client-statistics` y `/system-load`. Hoy recorren todos los datos, y la respuesta crecerá a medida que la base tenga más meses.
- Un endpoint de salud y el registro de cada petición con su tiempo de respuesta, para saber si la API está funcionando y detectar consultas lentas.